

1. Rôle et Architecture de Base

◆ Backend vs Frontend

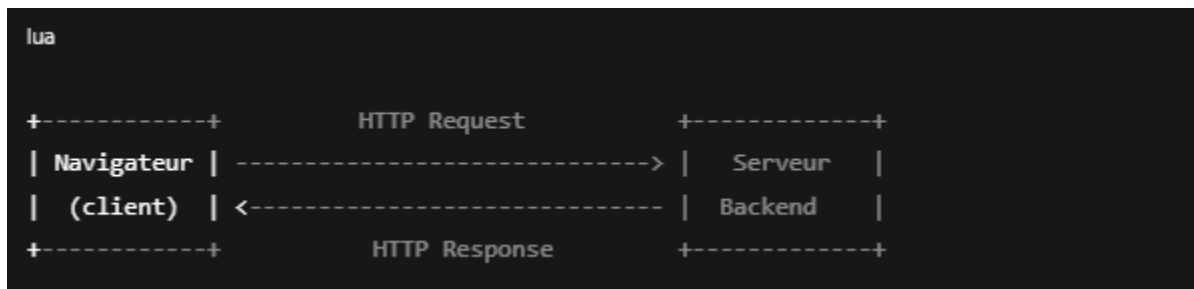
Frontend (client)

- ➡ Partie visible d'un site (HTML, CSS, JS).
- ➡ Sert à l'affichage et à l'interaction utilisateur.

Backend (serveur)

- ➡ Partie invisible.
- ➡ Gère :
 - la **logique métier**,
 - la **gestion des données**,
 - la **sécurité**,
 - l'**authentification**,
 - la **communication avec la base de données**.

◆ Schéma Client ↔ Serveur



Le backend reçoit des requêtes, traite les données, interroge la base, applique la logique métier, puis répond au client.

◆ Langages populaires du Backend

1. JavaScript → Node.js

- Très rapide
- Basé sur un moteur V8
- Idéal pour applications temps réel

2. PHP

- Très utilisé en web classique
- Simple à déployer
- Ex : WordPress, Laravel

3. Python

- Clair et lisible
- Ex : Django, Flask
- Très bon pour IA, data et API rapides

2. Gestion des Données (Bases de Données)

Le backend gère le **stockage permanent**.

◆ Bases SQL (Relationnelles) — ex : MySQL, PostgreSQL

- Données structurées
- Relations entre tables
- Utilisent le langage SQL
- Excellentes pour :
 - ✓ systèmes bancaires
 - ✓ stock, ERP
 - ✓ données fiables

Exemple de table SQL :

```
sql

TABLE Utilisateurs
-----
id | nom | email
1  | Ali | ali@mail.com
2  | Sara | sara@mail.com
```

♦ Bases NoSQL (Non-Relationnelles) — ex : MongoDB

- Données non structurées (documents JSON)
- Très flexibles
- Excellentes pour :
 - ✓ applications rapides
 - ✓ réseaux sociaux
 - ✓ données massives

Exemple document MongoDB :

```
json

{
  "nom": "Ali",
  "email": "ali@mail.com",
  "roles": ["user", "admin"]
}
```

♦ SQL vs NoSQL — Quand choisir ?

Situation	SQL	NoSQL
Données structurées	✓	
Beaucoup de relations	✓	
Performance haute + flexibilité		✓
Scalabilité horizontale		✓
Prototypage rapide		✓

◆ ORM / ODM

Un **ORM** (Object-Relational Mapping) ↔ relie le code aux données.

Exemples :

- **Node.js → Sequelize** (SQL)
- **Node.js → Mongoose** (MongoDB)
- **Python → SQLAlchemy**

Avantage : écrire du SQL sans écrire de SQL.

Exemple Sequelize :

```
js
User.findAll();
```

3. Les API et le Protocole HTTP

◆ Une API, c'est quoi ?

Une API est un **point d'accès** permettant à deux systèmes de communiquer.

Exemple dans un site :

- Le Frontend demande les produits
- Le Backend renvoie les données en JSON

◆ API REST : définition

REST → architecture permettant :

- des URL simples
- une communication claire
- des réponses structurées

Exemple d'URL REST :

```
bash
GET /api/produits
POST /api/produits
DELETE /api/produits/25
```

◆ Les 4 verbes HTTP principaux

Verbe	Action	CRUD
GET	Lire	Read
POST	Créer	Create
PUT/PATCH	Modifier	Update
DELETE	Supprimer	Delete

Exemple :

4. Sécurité et Authentification

◆ Authentification vs Autorisation

Concept	Signification
Authentification	Qui es-tu ? (login / token)
Autorisation	Que peux-tu faire ? (droits, accès)

◆ Méthode 1 : Sessions & Cookies

- Le backend stocke un **cookie de session**
- Traditionnel, sécurisé

◆ Méthode 2 : Tokens JWT

- Le backend donne un **token chiffré**
- Le frontend le renvoie dans chaque requête
- Ultra utilisé pour les API modernes

Exemple d'un JWT :

```
arduino
```

```
eyJhbGciOi... (très long)
```

5. Frameworks Backend

◆ Node.js

- **Express.js** (le plus utilisé)
- **NestJS** (structure MVC, très pro)

◆ Python

- **Django** (complet, batteries incluses)
- **Flask** (minimal, pour API simples)

◆ Pourquoi utiliser un framework ?

- ✓ Gagne du temps
- ✓ Sécurise le projet
- ✓ Structure le code
- ✓ Fournit des outils prêts à l'emploi :

- routing
- middleware
- sécurité
- gestion des erreurs
- templates

Sans framework = tout coder à la main.

Conclusion

« Le backend est la colonne vertébrale d'une application.

Il gère les données, la sécurité, la logique métier et la communication avec le frontend.

Grâce aux langages, frameworks, bases de données et API REST, il permet de fournir des services rapides, sécurisés et fiables. »